

ΜΥΕ023 – Παράλληλα Συστήματα και Προγραμματισμός

Σετ Ασκήσεων #2 (GPU programming)

Αθανάσιος Φουρκιώτης, AM 4940 cs04940@uoi.gr

Σε αυτή την εργασία έκανα δύο πράγματα. Πρώτα έγραψα έναν πολλαπλασιαστή πινάκων σε CUDA. Έπειτα έβαλα το φίλτρο Sobel να τρέχει στη GPU μέσω OpenMP offloading. Στόχος μου δεν ήταν μόνο να βγάλω καλούς χρόνους, αλλά και να σιγουρευτώ ότι τα αποτελέσματα είναι σωστά. Αλλιώς οι χρόνοι δεν λένε τίποτα. Όλα όσα γράφω παρακάτω είναι από τα δικά μου τρεξίματα στο parallax. Όλες οι μετρήσεις έγιναν στο parallax.

Παράμετρος	Τιμή
Όνομα μηχανήματος	parallax (gatepc73.cs.uoi.gr)
GPU	NVIDIA Tesla P40 (30 SM × 128 cores = 3840 cores)
CUDA toolkit	12.4
Μεταγλωττιστής CUDA	nvcc -O2
Μεταγλωττιστής OpenMP	clang 11 (-fopenmp -fopenmp-targets=nvptx64-nvidia-cuda)
Χρονομέτρηση	gettimeofday() και omp_get_wtime()

Πώς έτρεξα τα πειράματα

Κάθε διαμόρφωση την έτρεξα τέσσερις φορές και κρατάω τον μέσο όρο. Πριν προηγείται ένα warm-up run που δεν προσμετράται. Το έβαλα γιατί η πρώτη εκτέλεση μου έβγαζε αισθητά μεγαλύτερο χρόνο. Μάλλον οφείλεται σε αρχικοποίηση του CUDA context και JIT compilation. Αν την μετρούσα κανονικά, θα χάλαγε ο μέσος όρος. Οι συνδυασμοί που έπρεπε να δοκιμάσω ήταν αρκετοί, οπότε έγραψα ένα μικρό shell script — το run_experiments.sh — που κάνει compile και τρέχει όλα μαζί. Στο Sobel τρέχω και τις τρεις εικόνες, παρόλο που στους πίνακες αναφέρω αναλυτικά μόνο τη μεγαλύτερη, όπως ζητάει η εκφώνηση. Όλα τα νούμερα προέρχονται από το αρχείο results.txt.

Το flow που ακολούθησα για να τρέξω στο parallax ήταν περίπου το εξής:

τοπικό PowerShell

```
scp -P 2229 matrix-mul.c sobel.c run_experiments.sh \  
*.txt *.bmp ex2006@gatepc73.cs.uoi.gr:~/  
ssh ex2006@gatepc73.cs.uoi.gr -p 2229
```

μέσα στο parallax

```
chmod +x run_experiments.sh  
bash run_experiments.sh > results.txt 2>&1
```

τοπικό PowerShell - κατέβασμα αποτελεσμάτων

```
scp -P 2229 ex2006@gatepc73.cs.uoi.gr:~/results.txt .
```

Άσκηση 1 – Πολλαπλασιασμός πινάκων με CUDA

Το πρόβλημα

Στην πρώτη άσκηση ζητείται πολλαπλασιασμός πινάκων στη GPU με CUDA. Σαν αφετηρία πήρα το σειριακό πρόγραμμα από την ιστοσελίδα του μαθήματος. Στην έκδοσή μου, οι A και B στέλνονται στη μνήμη της GPU, εκτελείται το kernel, και ο C γυρνάει πίσω στον host για τον έλεγχο. Η υπογραφή του kernel είναι αυτή που ζητάει η εκφώνηση:

```
__global__ void matmul(int *A, int *B, int *C, int N);
```

Πώς το παραλληλοποίησα

Ξεκίνησα με τη μάλλον προφανή υλοποίηση κάθε thread υπολογίζει ένα στοιχείο του C. Κάθε thread διαβάζει επανειλημμένα τα ίδια δεδομένα από την global memory. Έτσι προχώρησα στην tiled έκδοση με shared memory. Εκεί κάθε block φορτώνει μικρά κομμάτια του A και του B στη shared memory, τα threads κάνουν τους πολλαπλασιασμούς πάνω στα κοινά δεδομένα, και προχωράμε στο επόμενο tile. Για το tile χρησιμοποίησα μέγεθος 16 και ορθογώνια blocks ώστε να ταιριάζουν με τα threads/block της εκφώνησης: 16x8 για 128 threads, 16x16 για 256, 16x32 για 512. Το grid προκύπτει αυτόματα από το N και το πλήθος των threads. Για παράδειγμα, για N=1024 με 128 threads/block βγαίνουν $1024*1024/128 = 8192$ blocks. Είναι ακριβώς το παράδειγμα που αναφέρει και η εκφώνηση. Όσον αφορά το memory access pattern, από όσο κατάλαβα τα writes στον C βγαίνουν coalesced γειτονικά threads πέφτουν σε γειτονικές θέσεις, οπότε η μνήμη γράφεται σε συνεχόμενα κομμάτια. Δεν είμαι 100% σίγουρος αν εκμεταλλεύομαι πλήρως τη bandwidth, αλλά οι χρόνοι μου έδειξαν ότι κάτι δουλεύει σωστά. Τον A τώρα, τον φορτώνω μία φορά ανά tile και τον ξαναχρησιμοποιούν όλα τα threads του block. Νομίζω ότι σε αυτό κυρίως οφείλεται το κέρδος έναντι της απλής έκδοσης που είχα δοκιμάσει αρχικά.

Το kernel που χρησιμοποίησα (από το αρχείο matrix-mul.c) είναι το παρακάτω:

```
__global__ void matmul(int *A, int *B, int *C, int n)
{
    __shared__ int As[BLOCK_ROWS][TILE_SIZE];
    __shared__ int Bs[TILE_SIZE][TILE_SIZE];

    int row = blockIdx.y * BLOCK_ROWS + threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + threadIdx.x;
    int sum = 0;

    for (int tile = 0; tile < n; tile += TILE_SIZE) {
```

```

    int a_col = tile + threadIdx.x;
    if (row < n && a_col < n)
        As[threadIdx.y][threadIdx.x] = A[row * n + a_col];
    else
        As[threadIdx.y][threadIdx.x] = 0;

    for (int ly = threadIdx.y; ly < TILE_SIZE; ly += BLOCK_ROWS) {
        int b_row = tile + ly;
        if (b_row < n && col < n)
            Bs[ly][threadIdx.x] = B[b_row * n + col];
        else
            Bs[ly][threadIdx.x] = 0;
    }

    __syncthreads();

    for (int k = 0; k < TILE_SIZE; k++)
        sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];

    __syncthreads();
}

if (row < n && col < n)
    C[row * n + col] = sum;
}

```

Για την επαλήθευση κάνω δύο ελέγχους. Πρώτα συγκρίνω τον C που γύρισε η GPU με τον C που υπολογίζω σειριακά στον host. Μετά συγκρίνω και με τα έτοιμα Cmat<N>.txt που μας δίνει η εκφώνηση. Δεν ήθελα ο έλεγχος να στηρίζεται μόνο στον δικό μου σειριακό κώδικα. Στις CUDA κλήσεις έχω μια check_cuda() που τερματίζει το πρόγραμμα αν αποτύχει κάποια κλήση: cudaMalloc, cudaMemcpy, kernel launch ή cudaFree. Για N=2048 οι πίνακες δεν χωρούν στο stack, οπότε τους δήλωσα static globals. Στον χρόνο της GPU συμπεριλαμβάνω και τις δύο μεταφορές host↔device. Νομίζω πως αυτό αντιπροσωπεύει καλύτερα το πραγματικό κόστος ενός χρήστη που τρέχει το πρόγραμμα από την αρχή ως το τέλος.

Μεταγλώττιση

```

nvcc -O2 -x cu -DN=512 -DTHREADS=128 matrix-mul.c -o matrix-mul
nvcc -O2 -x cu -DN=512 -DTHREADS=256 matrix-mul.c -o matrix-mul
... (ομοίως για κάθε συνδυασμό N × THREADS)

```

Αποτελέσματα μετρήσεων

Στους πίνακες που ακολουθούν δίνω τις τέσσερις μετρήσεις για κάθε διαμόρφωση μαζί με τον μέσο όρο. Το warm-up προηγείται όπως ανέφερα, αλλά δεν προσμετράται στον υπολογισμό. Ο σειριακός χρόνος στον host μπαίνει στους πίνακες για να φανεί το speedup. Πριν προχωρήσω, αξίζει να σημειώσω ότι οι μετρήσεις παρουσιάζουν μια μικρή φυσική διακύμανση από run σε run, της τάξης των μερικών microseconds. Αυτό είναι αναμενόμενο σε ένα κοινόχρηστο μηχάνημα, και ο μέσος όρος τεσσάρων μετρήσεων εξομαλύνει αρκετά την εικόνα. Στους πίνακες 1, 2 και 3 αντιστοιχούν τα τρία μεγέθη πινάκων που ζητάει η εκφώνηση, και στο τέλος υπάρχει και ένας συγκεντρωτικός πίνακας speedups για ευκολότερη ανάγνωση.

Πίνακας 1: CUDA matmul – $N = 512$ (χρόνοι σε sec)

Threads	Blocks	Serial	1η	2η	3η	4η	M.O.	Speedup
128	2048	0.126240	0.001166	0.001158	0.001156	0.001166	0.001161	108.69x
256	1024	0.124775	0.001156	0.001120	0.001122	0.001125	0.001131	110.34x
512	512	0.118165	0.001164	0.001145	0.001149	0.001134	0.001148	102.93x

Πίνακας 2: CUDA matmul – $N = 1024$ (χρόνοι σε sec)

Threads	Blocks	Serial	1η	2η	3η	4η	M.O.	Speedup
128	8192	3.078093	0.004872	0.004766	0.004779	0.004782	0.004800	641.30x
256	4096	3.066269	0.004732	0.004523	0.004565	0.004519	0.004585	668.80x
512	2048	3.060791	0.004951	0.004781	0.004761	0.004761	0.004813	635.88x

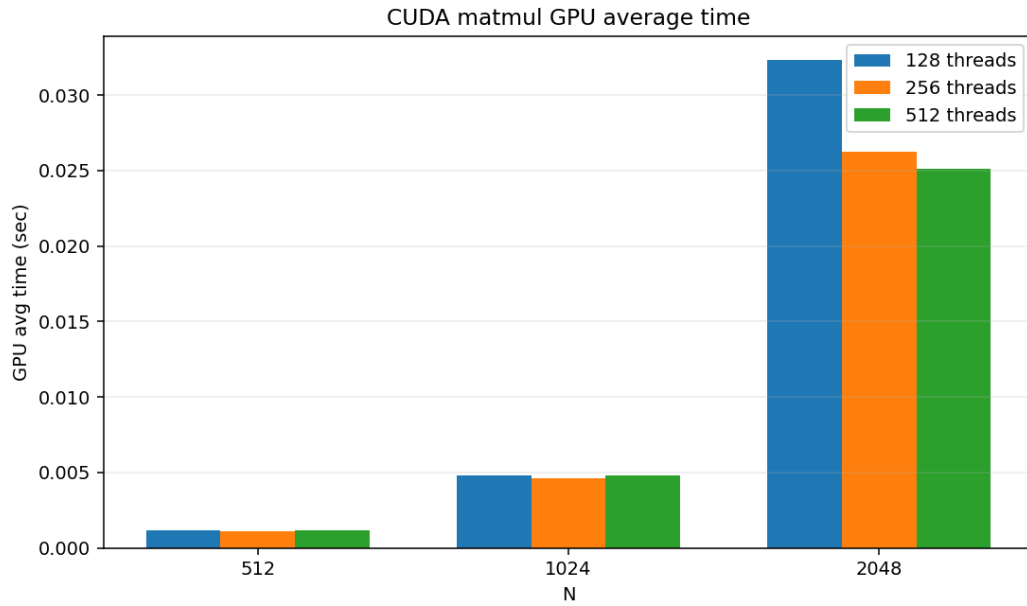
Πίνακας 3: CUDA matmul – $N = 2048$ (χρόνοι σε sec)

Threads	Blocks	Serial	1η	2η	3η	4η	M.O.	Speedup
128	32768	24.535971	0.032538	0.032437	0.032392	0.031837	0.032301	759.60x
256	16384	24.489216	0.026492	0.026459	0.026377	0.025633	0.026240	933.27x
512	8192	24.492629	0.025147	0.025097	0.025106	0.025088	0.025109	975.43x

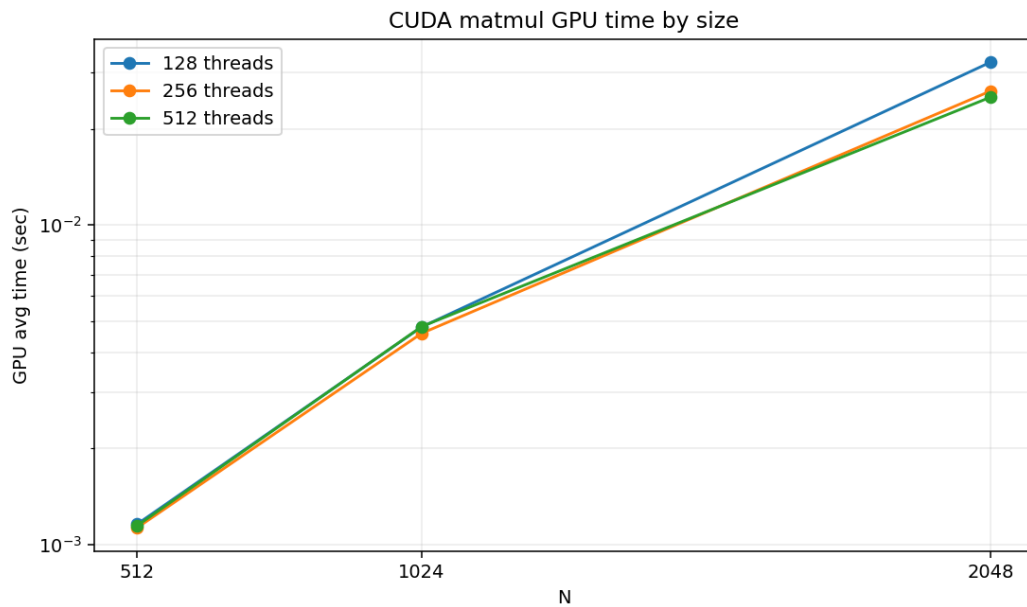
Συγκεντρωτικά speedups (μέσοι όροι):

N \ Threads	128	256	512
512	108.69x	110.34x	102.93x
1024	641.30x	668.80x	635.88x
2048	759.60x	933.27x	975.43x

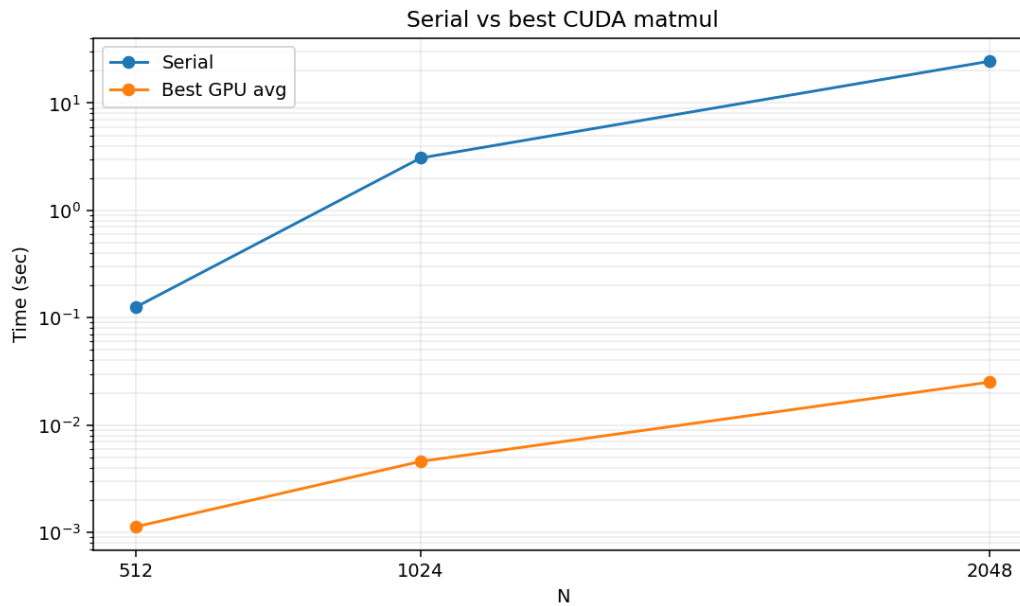
Γραφικές παραστάσεις



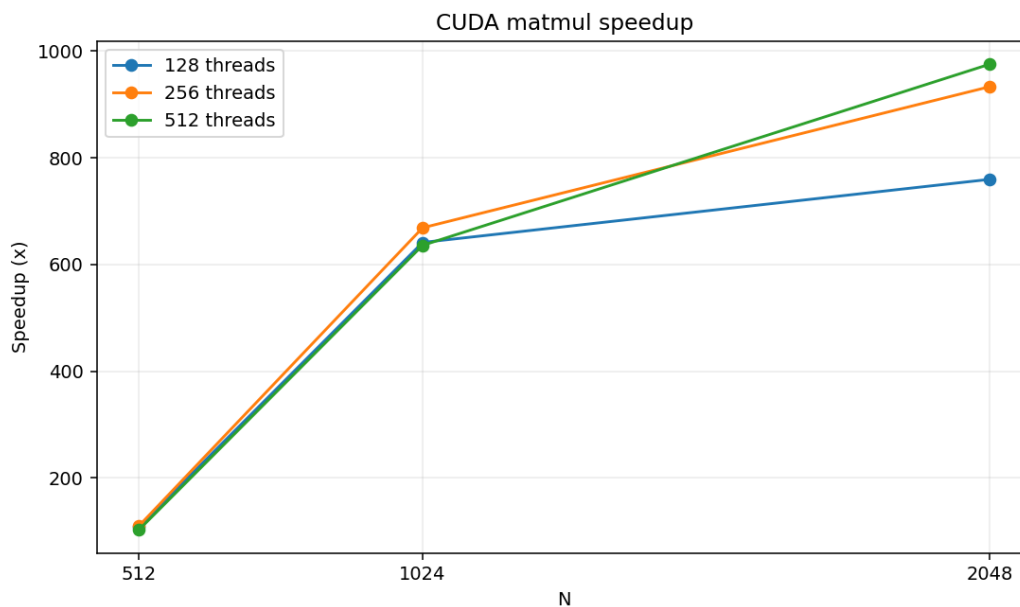
Σχήμα 1: μέσος χρόνος GPU ανά N και threads/block. Στο γράφημα φαίνεται καθαρά πως ο χρόνος είναι πολύ μικρός για $N=512$ και $N=1024$, και ανεβαίνει αισθητά μόνο στο $N=2048$.



Σχήμα 2: μέσος χρόνος GPU σε line plot με logarithmic άξονα y . Διάλεξα logarithmic κλίμακα γιατί ο χρόνος ανεβαίνει κατά τάξεις μεγέθους όσο μεγαλώνει το N , και σε γραμμική κλίμακα δεν θα φαινόντουσαν οι διαφορές στα μικρά N .



Σχήμα 3: σύγκριση σειριακού χρόνου και χρόνου GPU. Είναι ίσως το πιο εντυπωσιακό από τα γραφήματα, γιατί δείχνει ότι η σειριακή καμπύλη ανεβαίνει σχεδόν εκθετικά ενώ η GPU παραμένει σχεδόν επίπεδη. Παρόλο που ο πραγματικός λόγος είναι $O(N^3)$ και για τις δύο, η GPU κρύβει αυτή τη συμπεριφορά πίσω από την παραλληλία.



Σχήμα 4: speedup ανά μέγεθος πίνακα και threads/block. Παρατηρώ ότι το speedup ανεβαίνει συστηματικά καθώς μεγαλώνει το N , και πως οι τρεις καμπύλες δεν διαφέρουν δραματικά μεταξύ τους — εκτός από το $N=2048$, όπου το $THREADS=128$ πέφτει λίγο πιο πίσω από τις άλλες δύο.

Σχόλια στα αποτελέσματα

Σε όλες τις εννέα διαμορφώσεις και οι δύο έλεγχοι βγήκαν OK τόσο σε σχέση με τη σειριακή μου έκδοση όσο και με τα Cmat<N>.txt. Άρα το αποτέλεσμα είναι σωστό. Από τα νούμερα φαίνεται καθαρά ότι η GPU κερδίζει πολύ σε σχέση με αυτό του σειριακού. Ο σειριακός μου κώδικας είναι ο

απλός $O(N^3)$ χωρίς καμία βελτιστοποίηση. Ο πολλαπλασιασμός πινάκων είναι σχεδόν τέλεια παράλληλο πρόβλημα. Τα νούμερα είναι λογικά για τη συγκεκριμένη σύγκριση, αλλά δεν γενικεύονται. Στο $N=512$ ο σειριακός χρόνος είναι ήδη μικρός, οπότε τα overheads της GPU (μεταφορές και τα λοιπά) γίνονται πιο αισθητά. Στο $N=1024$ η ισορροπία βελτιώνεται. Στο $N=2048$ ο σειριακός κώδικας υποφέρει από cache misses, πιστεύω, οπότε το speedup μεγαλώνει ακόμα περισσότερο. Η αλλαγή των threads/block δεν επηρεάζει δραματικά την επίδοση, ειδικά στο $N=2048$ όπου οι τρεις τιμές είναι αρκετά κοντά. Για αυτό το kernel φαίνεται ότι πιο σημαντικό είναι το μέγεθος του προβλήματος και η συμπεριφορά της μνήμης παρά το ίδιο το block size. Αν ήθελα ακόμα καλύτερους χρόνους, το επόμενο βήμα δεν θα ήταν απλά να πειράξω το THREADS. Θα δοκίμαζα διαφορετικό tile size, περισσότερα στοιχεία ανά thread, ή στην ακραία περίπτωση σύγκριση με έτοιμη βιβλιοθήκη τύπου cuBLAS. Για τη συγκεκριμένη άσκηση όμως η tiled έκδοση που έγραψα φτάνει. Με δυσκόλεψε λίγο το πώς να ορίσω το 2D shape του block μαζί με το TILE_SIZE. Δοκίμασα και 32×4 και 16×8 πριν καταλήξω σε αυτό που έχω. Με μικρότερο tile οι χρόνοι μου έβγαιναν χειρότεροι. Δεν επέμεινα όμως στη βελτιστοποίηση γιατί τα νούμερα ήταν ήδη αρκετά καλά.

Άσκηση 2 – Φίλτρο Sobel στη GPU με OpenMP offloading

Το πρόβλημα

Στη δεύτερη άσκηση εφαρμόζω το φίλτρο Sobel σε εικόνες BMP. Το σειριακό πρόγραμμα μας δίνεται έτοιμο. Εγώ συμπλήρωσα τις δύο εκδόσεις που τρέχουν στη GPU με OpenMP offloading. Το φίλτρο για κάθε pixel χρησιμοποιεί τα δύο 3×3 kernels G_x και G_y , και βγάζει την ένταση του gradient ως $\sqrt{g_x^2 + g_y^2}$ ξεχωριστά για κάθε κανάλι R, G, B.

Πώς το παραλληλοποίησα

Έγραψα δύο εκδόσεις ακριβώς όπως ζητάει η εκφώνηση. Η μία βάζει όλα στην ίδια γραμμή (συνδυαστική οδηγία), η άλλη έχει εμφωλευμένη δομή (η μία οδηγία μέσα στην άλλη). Και οι δύο κάνουν τον ίδιο υπολογισμό, απλά μοιράζουν τη δουλειά στα teams και τα threads λίγο διαφορετικά.

Στην αρχή είχα βάλει collapse(2) και στις δύο εκδόσεις. Ήταν η εύκολη λύση. Όμως ο χρόνος της εμφωλευμένης έβγαινε σχεδόν ίδιος με της συνδυαστικής, οπότε δεν είχε νόημα να τις δίνω και τις δύο. Αφαίρεσα το collapse μόνο από τη nested και τότε εμφανίστηκε μια μικρή αλλά σταθερή διαφορά — γύρω στο 10% στις μεσαίες διαμορφώσεις. Δεν είμαι σίγουρος γιατί ακριβώς συμβαίνει αυτό. Η εξήγηση που δίνω παρακάτω (data locality) είναι αυτή που θεωρώ πιθανότερη. Δεν αποκλείω όμως να παίζει ρόλο και το πώς μοιράζει ο clang τα iterations.

Η συνδυαστική έκδοση (sobel_omp_device) είναι η παρακάτω:

```
#pragma omp target teams distribute parallel for collapse(2) \
    map(to: in_r[0:total], in_g[0:total], in_b[0:total]) \
    map(to: Gx[0:9], Gy[0:9]) \
```

```

    map(from: out_r[0:total], out_g[0:total], out_b[0:total]) \
    num_teams(NUM_TEAMS) thread_limit(THREAD_LIMIT)
for (int i = 0; i < height; i++) {
    for (int j = 0; j < width; j++) {
        ...
    }
}

```

Με το `collapse(2)` ο μεταγλωττιστής βλέπει έναν μεγάλο επίπεδο χώρο επαναλήψεων (περίπου 2.25 εκατομμύρια pixels για την εικόνα 1500×1500) και τον μοιράζει σε `teams × threads` χωρίς να ξεχωρίζει γραμμές από στήλες.

Η εμφωλευμένη έκδοση (`sobel_omp_device_nested`) είναι η εξής:

```

#pragma omp target map(to:...) map(from:...)
{
    #pragma omp teams distribute num_teams(NUM_TEAMS)
    thread_limit(THREAD_LIMIT)
    for (int i = 0; i < height; i++) {
        #pragma omp parallel for
        for (int j = 0; j < width; j++) {
            ...
        }
    }
}

```

Ο εξωτερικός βρόχος (γραμμές της εικόνας) μοιράζεται στα `teams`, ενώ ο εσωτερικός (στήλες) στα `threads` κάθε `team`. Δεν έβαλα `collapse(2)` εδώ επίτηδες για να υπάρχει πραγματική διαφορά από την πρώτη έκδοση. Στις εκδόσεις το `target` ζητάει εκτέλεση στη συσκευή, το `map(to/from)` μεταφέρει τα δεδομένα από και προς τη GPU, και τα `num_teams / thread_limit` τα ορίζω από το `compile` (μέσω `-DNUM_TEAMS` και `-DTHREAD_LIMIT`). Έτσι τρέχω εύκολα πολλούς συνδυασμούς από το `script`. Σαν βάση για τους συνδυασμούς πήρα τα 3840 cores της P40 (30 SM × 128 cores). Δεν λεώ ότι το OpenMP δουλεύει ένα προς ένα με τους cores, αλλά είναι ένας τρόπος να επιλέξω πόσα `teams` και `threads/team` θα δοκιμάσω για να μη μένουν SM χωρίς κάποια δουλειά. Ο περιορισμός ότι το `thread_limit` είναι πολλαπλάσιο του 32 τηρείται παντού. Δύο λεπτομέρειες το `clamp` στα όρια της εικόνας το έγραψα inline μέσα στο `target region` και όχι ως εξωτερική συνάρτηση γιατί έτσι δούλεψε καθαρά με το offloading. Και το `thread_limit` το δίνω στην `teams` οδηγία, επειδή στο `parallax` ο clang 11 ήταν πιο σταθερός με αυτή την επιλογή.

Επιβεβαίωση ότι το offloading είναι ενεργό

Για να βεβαιωθεί ότι το πρόγραμμα όντως τρέχει στη GPU και όχι σε host fallback, πρόσθεσα τη συνάρτηση `print_offloading_status()`. Τυπώνει πόσες συσκευές βλέπει το OpenMP και τι επιστρέφει η `omp_is_initial_device()` μέσα σε ένα target region.

```
void print_offloading_status(void)
{
    int num_devs = omp_get_num_devices();
    int in_target_says_initial = -1;

    #pragma omp target map(tofrom: in_target_says_initial)
    {
        in_target_says_initial = omp_is_initial_device();
    }
    /* τυπώνουμε και τα δύο */
}
```

Στα τελικά πειράματα η `omp_get_num_devices()` γυρίζει 1 και η `omp_is_initial_device()` μέσα στο target γυρίζει 0. Δηλαδή η εκτέλεση πραγματικά γίνεται στη συσκευή και όχι στον host.

Διαμορφώσεις που δοκίμασα

Δοκίμασα όλες τις διαμορφώσεις όπου $\text{threads} \times \text{teams} \approx 3840$ και threads πολλαπλάσιο του 32. Πρόσθεσα και δύο επιπλέον (512×8 και 1024×4), όπου το γινόμενο φτάνει τα 4096, για να δω αν αλλάζει κάτι όταν ξεπερνάμε λίγο τα cores. Τις παρουσιάζω χωριστά και δεν τις λογαριάζω σαν βασικές.

Μεταγλώττιση

```
clang -O2 -fopenmp -fopenmp-targets=nvptx64-nvidia-cuda \
    -DNUM_TEAMS=$TEAMS -DTHREAD_LIMIT=$THREADS \
    sobel.c -lm -o sobel
```

Σχετικά με τα warnings

Στην εκτέλεση εμφανίστηκαν κάποια warnings τα οποία θέλω να αναφέρω, ώστε να μη φαίνεται ότι τα αγνόησα. Το OpenMP runtime έβγαζε ένα warning για host-side teams («Cannot form a team with 120 threads...»). Δεν αφορά την GPU εκτέλεση, αλλά γέμιζε το output. Για να σταματήσει, πρόσθεσα στο script τα εξής πριν τα τρεξίματα:

```
export OMP_NUM_THREADS=1
export OMP_MAX_ACTIVE_LEVELS=2
unset KMP_DEVICE_THREAD_LIMIT
unset KMP_TEAMS_THREAD_LIMIT
```

```
unset OMP_THREAD_LIMIT
```

Στα δικά μου τρεξίματα παρέμειναν μερικά `rtxas warnings` και ένα `warning` του `clang` σχετικά με την έκδοση της `CUDA`. Νομίζω ότι το δεύτερο εμφανίζεται επειδή ο `clang 11` στο `parallax` δεν αναγνωρίζει εντελώς τη `CUDA 12.4`. Εφόσον το εκτελέσιμο παράγεται κανονικά και η εκτέλεση γίνεται στη `GPU`, όπως φαίνεται από την έξοδο του `omp_is_initial_device()` και τα θεώρησα αμελητέα. Τα `rtxas warnings` από ό,τι κατάλαβα διαβάζοντας τα μηνύματα αναφέρονται σε `hints` για τα `target kernels`, και δεν παρατήρησα κάποια επίπτωση στην ορθότητα του αποτελέσματος. Σε όλα τα τρεξίματα που έκανα τα αποτελέσματα `Sobel` έμοιαζαν κοντά με αυτά της σειριακής έκδοσης.

Αποτελέσματα μετρήσεων

Το `script` τρέχει το `Sobel` και για τις τρεις εικόνες, δηλαδή `500x500`, `1000x1000` και `1500x1500` ώστε να καλύπτω όλο το εύρος των εισόδων που μας δόθηκαν. Παρόλα αυτά οι αναλυτικοί πίνακες που ακολουθούν αφορούν μόνο τη μεγαλύτερη εικόνα. Από τα τρεξίματά μου είδα ότι ο σειριακός χρόνος έμενε σταθερός, κάπου ανάμεσα στα `73` και `74 ms`, και αυτό μου έδωσε μια βάση για να βγάλω `speedups`. Συγκρίνω αυτόν τον σειριακό χρόνο με τους μέσους όρους των δύο εκδόσεων `GPU` (συνδυαστική και εμφωλευμένη), παίρνοντας `4` μετρήσεις για κάθε διαμόρφωση και έναν `warm-up run` που δεν προσμετράται. Οι πίνακες που σημειώνονται ως «4α» και «5α» αντιστοιχούν στις διαμορφώσεις των `3840 cores`, που είναι αυτές που ταιριάζουν με την περιγραφή της `P40` στην εκφώνηση. Στους πίνακες «4β» και «5β» βάζω χωριστά τις δύο επιπλέον διαμορφώσεις των `4096 logical threads` (`512x8` και `1024x4`), τις οποίες δοκίμασα μόνο και μόνο για να δω αν αλλάζει κάτι όταν ξεπερνάμε λίγο το πλήθος των `cores`.

Πίνακας 4α: *Sobel* – Συνδυαστική οδηγία – *3840 cores* (sec, M.O. 4 runs)

threads	teams	cores	Serial	GPU M.O.	Speedup
32	120	3840	0.073670	0.007456	9.88x
64	60	3840	0.073872	0.007517	9.83x
96	40	3840	0.074004	0.007642	9.68x
128	30	3840	0.073932	0.007557	9.78x
160	24	3840	0.073775	0.007647	9.65x
192	20	3840	0.074037	0.007913	9.36x
256	15	3840	0.073794	0.007970	9.26x
320	12	3840	0.074198	0.008533	8.70x
384	10	3840	0.073854	0.008609	8.58x
480	8	3840	0.074087	0.009117	8.13x
640	6	3840	0.073867	0.009788	7.55x
768	5	3840	0.078992	0.010265	7.70x
960	4	3840	0.073155	0.012634	5.79x

Πίνακας 4β: *Sobel* – Συνδυαστική οδηγία – *4096 cores* (έξτρα)

threads	teams	cores	Serial	GPU M.O.	Speedup
512	8	4096	0.073911	0.008817	8.38x
1024	4	4096	0.073500	0.012615	5.83x

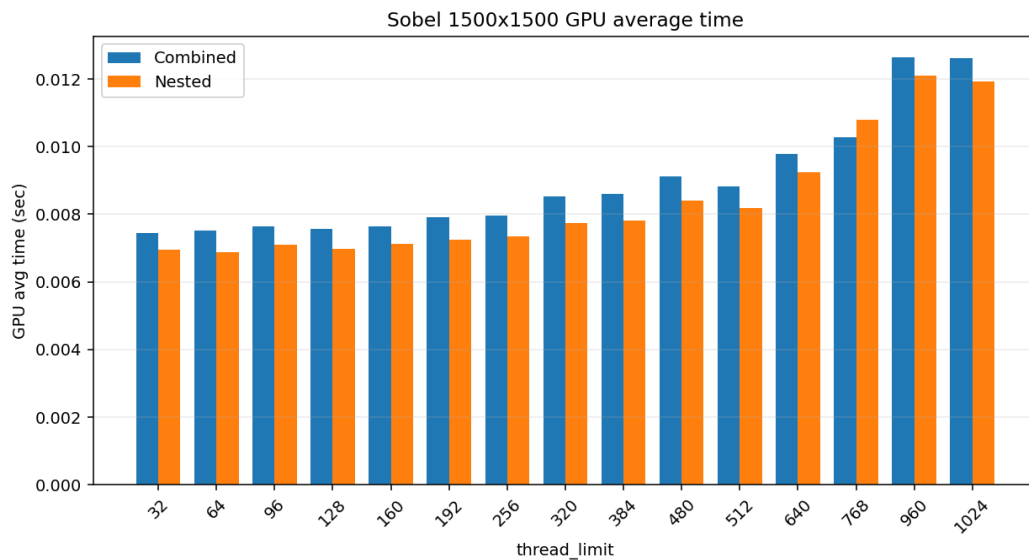
Πίνακας 5α: Sobel – Εμφωλευμένες οδηγίες – 3840 cores (sec, M.O. 4 runs)

threads	teams	cores	Serial	GPU M.O.	Speedup
32	120	3840	0.073670	0.006952	10.60x
64	60	3840	0.073872	0.006885	10.73x
96	40	3840	0.074004	0.007092	10.43x
128	30	3840	0.073932	0.006965	10.62x
160	24	3840	0.073775	0.007132	10.34x
192	20	3840	0.074037	0.007241	10.22x
256	15	3840	0.073794	0.007337	10.06x
320	12	3840	0.074198	0.007752	9.57x
384	10	3840	0.073854	0.007812	9.45x
480	8	3840	0.074087	0.008395	8.83x
640	6	3840	0.073867	0.009251	7.98x
768	5	3840	0.078992	0.010784	7.32x
960	4	3840	0.073155	0.012106	6.04x

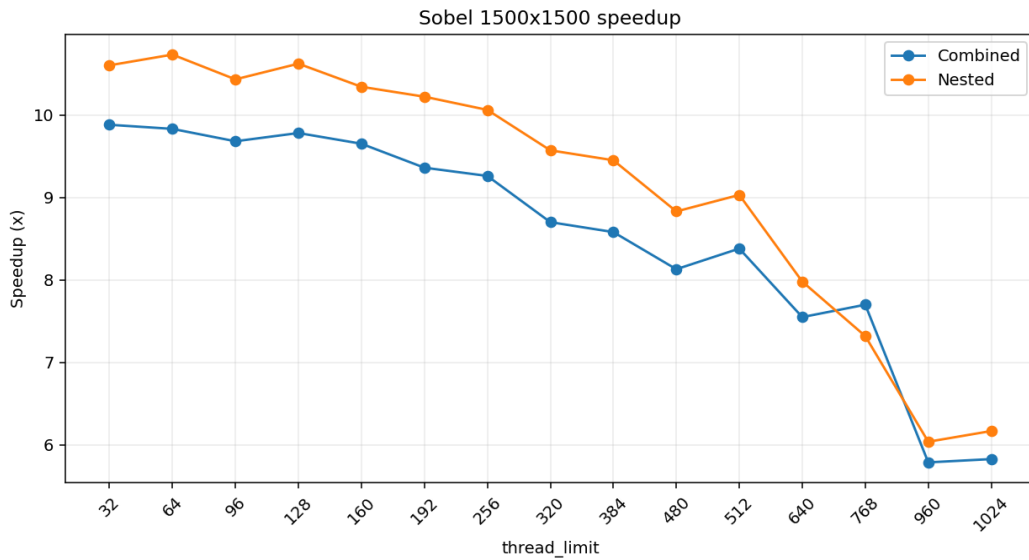
Πίνακας 5β: Sobel – Εμφωλευμένες οδηγίες – 4096 cores (έξτρα)

threads	teams	cores	Serial	GPU M.O.	Speedup
512	8	4096	0.073911	0.008182	9.03x
1024	4	4096	0.073500	0.011919	6.17x

Γραφικές παραστάσεις



Σχήμα 5: μέσοι χρόνοι GPU για τη συνδυαστική και την εμφωλευμένη έκδοση.



Σχήμα 6: speedup για τη συνδυαστική και την εμφωλευμένη έκδοση.

Σχόλια στα αποτελέσματα

Σε όλα τα Sobel πειράματα η `compare_images()` βγήκε OK, δηλαδή τόσο η συνδυαστική όσο και η εμφωλευμένη έκδοση παρήγαγαν την ίδια εικόνα με τη σειριακή. Σε συνδυασμό με το «in-target `omp_is_initial_device() = 0`» που εμφανίζεται στην έξοδο, η εκτέλεση στη GPU είναι επιβεβαιωμένη.

Ο καλύτερος χρόνος GPU βγαίνει στην εμφωλευμένη έκδοση με `thread_limit = 64` και `num_teams = 60` (περίπου 6.89 ms, speedup 10.73x). Πολύ κοντά βρίσκεται και η διαμόρφωση 128×30 . Το γενικό συμπέρασμα είναι ότι οι μεσαίες διαμορφώσεις με αρκετά teams δίνουν τη σταθερότερη επίδοση. Σε γενικές γραμμές η εμφωλευμένη έκδοση βγαίνει λίγο πιο γρήγορη από τη συνδυαστική στις περισσότερες περιπτώσεις. Η διαφορά δεν είναι μεγάλη αλλά είναι αρκετά συστηματική για να την αναφέρω. Μια πιθανή εξήγηση είναι ότι αφήνοντας τον εξωτερικό βρόχο στα teams και τον εσωτερικό στα threads, οι προσβάσεις γίνονται πιο γραμμικά και ταιριάζουν καλύτερα στο memory hierarchy της GPU. Όταν το `thread_limit` ανεβαίνει πολύ και τα teams μειώνονται αντίστοιχα ώστε το γινόμενο να μένει σταθερό, η επίδοση αρχίζει να πέφτει. Η εξήγηση είναι ότι με λίγα teams δεν γεμίζουν καλά όλα τα SM της P40. Με 4 teams, για παράδειγμα, μένουν πολλά SM χωρίς δουλειά. Από την άλλη, οι διαμορφώσεις με πολλά teams και μικρό ή μεσαίο `thread_limit` δουλεύουν καλά, με το βέλτιστο σημείο γύρω από 64 threads $\times$ 60 teams. Οι έξτρα διαμορφώσεις με 4096 logical threads δεν έδωσαν κάτι καθαρά καλύτερο. Τα κύρια συμπεράσματα τα βασίζω στις διαμορφώσεις των 3840, που είναι και αυτές που αντιστοιχούν στην περιγραφή της P40 στην εκφώνηση.

Συνολικά συμπεράσματα

Συγκεντρωτικά οι καλύτερες διαμορφώσεις που πέτυχα:

Πρόβλημα	Καλύτερη διαμόρφωση	Serial	GPU M.O.	Speedup
----------	---------------------	--------	----------	---------

Matmul N=512	THREADS=256	0.125 sec	1.13 ms	110.34x
Matmul N=1024	THREADS=256	3.066 sec	4.59 ms	668.80x
Matmul N=2048	THREADS=512	24.493 sec	25.11 ms	975.43x
Sobel 1500x1500	Nested 64 x 60	73.9 ms	6.89 ms	10.73x

Συνολικά στα δικά μου πειράματα η GPU κέρδισε αρκετά και στις δύο ασκήσεις. Στο matmul η tiled έκδοση με shared memory έδωσε πολύ μεγάλα speedups. Πρέπει όμως να μπει σε πλαίσιο: ο σειριακός κώδικας είναι ο απλός $O(N^3)$ που υποφέρει από cache misses στις μεγάλες διαστάσεις, και αυτό φουσκώνει το ratio. Στο Sobel η εικόνα είναι σχετικά μικρή και ο σειριακός κώδικας πιο γρήγορος ανά pixel, οπότε το speedup ήταν πιο μετρημένο — γύρω στο 10x. Για το matmul, η καλύτερη επιλογή threads/block στα τρεξίματά μου εξαρτήθηκε από το N. Κέρδισε το THREADS=256 για N=512 και N=1024, ενώ για N=2048 κέρδισε το THREADS=512. Νομίζω ότι το πιο ενδιαφέρον σημείο δεν είναι το block size αλλά η οργάνωση σε 2D blocks σε συνδυασμό με την επαναχρησιμοποίηση δεδομένων στη shared memory. Στο Sobel η εμφωλευμένη έκδοση κέρδισε στις περισσότερες περιπτώσεις, με σημαντικότερο παράγοντα να υπάρχουν αρκετά teams ώστε να δουλεύουν όλα τα SM. Το thread_limit από μόνο του δεν φάνηκε αρκετό, αν δεν συνοδευόταν από επαρκές πλήθος teams.

Στα τελικά τρεξίματα όλες οι επαληθεύσεις βγήκαν OK: το matmul ταυτίστηκε και με τη δική μου σειριακή έκδοση και με τα Cmat<N>.txt, ενώ το Sobel παρήγαγε την ίδια εικόνα με τη σειριακή έκδοση. Έβαλα και τους δύο ελέγχους σε κάθε ένα από τα 9 τρεξίματα του matmul, ώστε να μη χρειαστεί να ξανατρέξω αν αμφισβητηθεί κάτι. Άρα οι χρόνοι που αναφέρω προέρχονται από κανονική και σωστή εκτέλεση στη GPU, και όχι από μερικό ή λάθος υπολογισμό. Συνολικά είμαι ικανοποιημένος με το αποτέλεσμα και πιστεύω ότι καλύπτει όσα ζητάει η εκφώνηση.